```
while (line = gets)
  puts line.downcase
end
```

The assignment statement sets the variable line to the result of calling gets, which will either be the next line of text or nil. Then the while statement tests the value returned by the assignment statement, which is the value assigned. When the value is nil, that means the output has no further lines and the while loop terminates.

Ruby *statement modifiers* are a useful shortcut if the body of an if or while statement is just a single expression. Simply write the expression, followed by if or while and the condition. For example, here's a simple if statement:

```
if radiation > 3000
  puts "Danger, Will Robinson"
end
```

Here it is again, rewritten using a statement modifier:

```
puts "Danger, Will Robinson" if radiation > 3000
```

Similarly, this while loop:

```
square = 4
while square < 1000
  square = square * square
end
```

becomes this more concise version:

```
square = 4
square = square * square while square < 1000
```

The if version of these modifiers is perhaps most commonly used as a guard clause at the beginning of a method, as in return nil if user.nil?. The while version is much less commonly used.

Regular Expressions

Most of Ruby's built-in types will be familiar to all programmers. A majority of languages have strings, integers, floats, arrays, and so on. However not all languages have built-in support for regular expressions, the way that Ruby or JavaScript do. This is a shame, because regular expressions, although cryptic, are a powerful tool for working with text. And having them built in, rather than tacked on through a library interface, makes a big difference.

Entire books have been written about regular expressions (for example, *Mastering Regular Expressions* by Jeffrey Friedl), so we won't try to cover everything in this short section. Instead, we'll look at just a few examples of regular expressions in action. You'll find more coverage of regular expressions in Chapter 8, Regular Expressions, on page 129.

A regular expression is simply a way of specifying a pattern of characters to be matched in a string. In Ruby, you typically create a regular expression by writing a pattern between slash characters (*/pattern/*). And, Ruby being Ruby, regular expressions are objects and can be manipulated as such.